FULL STACK BANGALORE

Pertemuan 26

Spark Optimization & Performance Tuning

Sesi 16 • Fullstack Bangalore

Oleh Bayuzen Ahmad

©2025 Full Stack Bangalore

Contoh data

Pekerjaan yang akan kita jalankan

Query adalah rangkaian perintah untuk mengolah data. Contoh kita adalah ringkasan penjualan.

Data masukan

orders: daftar transaksi.
customers: daftar pelanggan dan wilayahnya.

Keduanya dibuat oleh kode latihan.

Hasil akhir

Lima baris, masing-masing untuk satu wilayah.

Setiap baris berisi total penjualan dan jumlah transaksi.

Notebook 5 / Script: orders, customers, aggregate()

©2025 Full Stack Bangalore

Istilah data

DataFrame, baris, kolom, dan key

DataFrame adalah bentuk tabel yang dapat diolah oleh Spark.

Istilah	Arti dan contoh di latihan
Baris / row	Satu catatan. Pada orders, satu baris mewakili satu transaksi.
Kolom / column	Jenis informasi, misalnya amount untuk nilai transaksi.
Key / kunci	Kolom untuk mencocokkan data. Kita memakai customer_id.
Schema / struktur	Nama dan tipe kolom, misalnya angka bulat atau teks.

Notebook 5 / Script: select(), alias(), cast()

©2025 Full Stack Bangalore

Tujuan optimasi

Apa yang dimaksud optimasi Spark?

Mengurangi pekerjaan atau waktu pemrosesan tanpa mengubah hasil yang dibutuhkan.

Optimization	Mengurangi pekerjaan, misalnya menghitung ulang data yang sama.
Tuning	Menyesuaikan pengaturan, misalnya jumlah bagian data yang diproses.
Ukuran keberhasilan	Hasil tetap benar. Waktu dan penggunaan resource diperiksa.

Notebook 7–16 / Script: one_trial(), summary

©2025 Full Stack Bangalore

Lingkungan praktik

Python, PySpark, Java, dan kernel

Keempat istilah ini muncul saat menyiapkan notebook.

Istilah	Perannya
Python	Bahasa untuk menulis langkah pengolahan data.
PySpark	Paket Python untuk menggunakan Spark. Versi lab: 4.0.1.
Java / JVM	Lingkungan yang menjalankan mesin Spark. Lab diuji dengan Java 17.
Kernel notebook	Proses Python yang menjalankan sel kode dan menyimpan variabel.

Notebook 2 / Script: pemeriksaan Java dan PySpark

©2025 Full Stack Bangalore

Persiapan

Instalasi paket pada environment lab

Jalankan di PowerShell dari folder sesi16. Buat .venv hanya jika belum tersedia.

```
py -m venv .venv

.\.venv\Scripts\python.exe -m pip install `
  -r lab\requirements-notebook.txt
```

.venv: folder Python khusus lab.

pip: alat pemasang paket Python.

-r: membaca daftar paket dari file.

Pilih Python .venv sebagai kernel.

Notebook petunjuk, 2 / Script: requirements.txt

©2025 Full Stack Bangalore

Cara menjalankan

Sel notebook dan perintah terminal

Keduanya menjalankan eksperimen Spark, dengan cara penggunaan berbeda.

Notebook .ipynb	Script .py
Kode dibagi menjadi sel.	Satu perintah memilih satu eksperimen.
Penjelasan dan hasil muncul di halaman yang sama.	Hasil muncul di terminal dan folder output.
Jalankan dari atas ke bawah.	Pakai --experiment untuk memilih latihan.
Restart Kernel lalu Run All untuk mengulang.	Jalankan perintah baru untuk membuat run baru.

Notebook 2–17 / Script: argparse / --experiment

©2025 Full Stack Bangalore

Pengaturan awal

Jumlah baris dan jumlah pengulangan

Varian adalah satu pilihan yang dibandingkan. Samakan data sebelum membandingkan waktu.

Pengaturan	Notebook	Script terminal
Jumlah data awal	N_ROWS = 10_000	--rows 300000 jika tidak diisi
Pengukuran tiap varian	REPEATS = 3	--repeats 3
Proses lokal	local[2]	--master local[2]
Contoh penyamaan	N_ROWS = 10_000	Tambahkan --rows 10000

Notebook 3–4 / Script: --rows, --repeats, --master

©2025 Full Stack Bangalore

Mesin Spark

Driver, executor, dan local[2]

Spark membagi data menjadi pekerjaan yang dapat dikerjakan bersamaan.

Driver dan executor

Driver mengatur aplikasi dan pekerjaan.

Executor menjalankan bagian pekerjaan serta menyimpan cache.

Yang terjadi pada lab

local[2] memakai dua thread kerja pada satu komputer.

Lab ini tidak membuat cluster beberapa komputer.

Notebook 4 / Script: SparkSession.builder

©2025 Full Stack Bangalore

Data sintetis

Cara kode membuat transaksi

Data sintetis adalah data buatan dengan pola yang dapat diulang.

```
orders = spark.range(N_ROWS).select(
    F.col("id").alias("order_id"),
    (F.col("id") % 1000).alias("customer_id"),
    ((F.col("id") % 97 + 1) * 100)
        .alias("amount")
)
```

range membuat nomor mulai dari 0.

% berarti sisa pembagian.

customer_id berulang dari 0–999.

amount adalah angka nilai transaksi.

Notebook 5 / Script: pembentukan orders

©2025 Full Stack Bangalore

Menggabungkan data

Join mencocokkan pelanggan pada transaksi

Kolom customer_id menjadi penghubung kedua tabel.

Data	Contoh baris	Maknanya
orders	customer_id=7, amount=800	Pelanggan 7 melakukan transaksi 800.
customers	customer_id=7, region=2	Pelanggan 7 berada di wilayah 2.
Sesudah join	customer_id=7, amount=800, region=2	Transaksi kini memiliki informasi wilayah.

Notebook 5, 8 / Script: orders.join(customers, ...)

©2025 Full Stack Bangalore

Meringkas data

Agregasi: jumlah penjualan per wilayah

Agregasi adalah perhitungan ringkasan dari beberapa baris.

```
df.groupBy("region").agg(
    F.sum("amount").alias("revenue"),
    F.count("*").alias("orders")
)
```

groupBy mengelompokkan menurut wilayah.

sum menjumlahkan nilai transaksi.

count menghitung jumlah transaksi.

revenue dan orders adalah nama hasil.

Notebook 5 / Script: aggregate()

©2025 Full Stack Bangalore

Waktu eksekusi

Transformation, action, dan lazy evaluation

Spark menyusun rencana terlebih dahulu, kemudian bekerja saat hasil diminta.

Transformation	Perintah seperti select, filter, dan join menyusun rencana pengolahan.
Action	Perintah seperti collect dan count meminta Spark menjalankan pekerjaan.
Lazy evaluation	Pengolahan data ditunda sampai action memerlukan hasilnya.

Notebook 6 / Script: q.collect()

©2025 Full Stack Bangalore

Action

Perbedaan show(), count(), dan collect()

Pilih action sesuai hasil yang ingin dibaca atau diukur.

Action	Hasil	Pemakaian pada notebook
show(5)	Menampilkan beberapa baris.	Melihat contoh data, bukan mengukur seluruh hasil.
count()	Menghasilkan jumlah baris.	Memeriksa filter dan mematerialisasi cache.
collect()	Mengambil hasil ke memori driver.	Mengambil ringkasan lima wilayah.

Notebook 5–6, 12, 14 / Script: collect(); cache: count()

©2025 Full Stack Bangalore

Pembagian data

Partisi adalah satu bagian dari dataset

Spark mengolah data melalui beberapa bagian agar pekerjaan dapat dibagi.

Contoh sederhana

Bayangkan 10.000 catatan dibagi menjadi beberapa tumpukan.

Satu partisi adalah satu tumpukan untuk dikerjakan.

Dampaknya

Terlalu sedikit: pekerjaan serentak terbatas.

Terlalu banyak: biaya mengatur pekerjaan bertambah.

Notebook 9–10 / Script: numPartitions; shuffle.partitions

©2025 Full Stack Bangalore

Pekerjaan Spark

Job, stage, dan task

Istilah ini digunakan pada halaman pemantauan Spark UI.

Istilah	Arti sederhana
Job	Rangkaian pekerjaan yang muncul karena action meminta hasil.
Stage	Satu tahap berisi kumpulan task dengan pekerjaan sejenis.
Task	Pekerjaan untuk satu partisi pada satu stage.
Paralelisme	Beberapa task dikerjakan bersamaan sesuai resource yang tersedia.

Notebook 6, 8–13 / Script: action dan Spark UI

©2025 Full Stack Bangalore

Pertukaran data

Shuffle mengelompokkan ulang data

Baris yang diperlukan bersama dapat berasal dari partisi yang berbeda.

Mengapa diperlukan

Transaksi wilayah 2 tersebar di beberapa partisi.

Untuk membuat total wilayah 2, hasil per bagian harus dipertemukan.

Biaya yang muncul

Data perlu dikirim, disusun, dan kadang ditulis sementara ke disk.

Pada plan, perpindahan dapat terlihat sebagai Exchange.

Notebook 6, 10 / Script: join dan groupBy

©2025 Full Stack Bangalore

Rencana query

Execution plan menjelaskan pekerjaan Spark

Operator adalah langkah yang muncul dalam rencana eksekusi.

Nama pada plan	Arti dalam latihan
Range / Scan	Sumber data. Lab memakai Range untuk membuat nomor.
Filter / Project	Memilih baris / memilih atau menghitung kolom.
Exchange / Sort	Mengatur ulang distribusi / mengurutkan data.
Join / HashAggregate	Menggabungkan tabel / menghitung ringkasan.

Notebook 6 / Script: save_plan(); explain("formatted")

©2025 Full Stack Bangalore

Baseline, kandidat, warm-up, dan trial

Baseline adalah kondisi awal. Kandidat adalah kondisi setelah perubahan.

```
started = perf_counter()
rows = query.collect()
seconds = perf_counter() - started

assert canonical(rows) == reference
```

Warm-up: eksekusi awal sebelum dicatat.

Trial: satu pengukuran yang dicatat.

Timer berhenti sebelum pengecekan hasil.

Median: nilai tengah dari tiga trial.

Notebook 7 / Script: one_trial(); perf_counter()

©2025 Full Stack Bangalore

Kebenaran hasil

Reference dan assertion

Reference adalah hasil pembanding. Assertion memeriksa apakah syarat terpenuhi.

Referensi notebook

Python menghitung total dari rumus transaksi, tanpa join Spark.

Hasilnya dibandingkan dengan hasil setiap varian.

Referensi script

Spark menghitung region langsung dari customer_id, tanpa join kandidat.

Kedua cara memakai pola data buatan yang sama.

Notebook 5, 15 / Script: reference; canonical(); assert

©2025 Full Stack Bangalore

Strategi join

Sort-merge join: urutkan lalu cocokkan

Strategi adalah cara Spark melakukan join.

Cara kerja

Data dibagi menurut key.
Setiap bagian diurutkan menurut key.
Baris dari kedua sisi kemudian dicocokkan.

Pemakaian di lab

Menjadi baseline ketika broadcast otomatis dan AQE dimatikan.

Cari SortMergeJoin serta Sort pada plan.

Notebook 8 / Script: join: baseline

©2025 Full Stack Bangalore

Strategi join

Broadcast join: salin sisi yang kecil

Tabel referensi kecil dapat dikirim agar pencocokan dilakukan pada task.

Kapan dipakai

Satu sisi join cukup kecil untuk memori executor.

Contoh: 1.000 pelanggan digabung dengan banyak transaksi.

Yang perlu diperhatikan

Penyalinan tetap memiliki biaya.
Tabel terlalu besar dapat menekan memori.

Hasil harus tetap sama dengan baseline.

Notebook 8 / Script: join: broadcast

©2025 Full Stack Bangalore

Kode join

Perubahan satu baris pada sisi pelanggan

Kedua varian memakai inner join dan ringkasan wilayah yang sama.

```
# Baseline
q1 = aggregate(orders.join(
    customers, "customer_id", "inner"))

# Kandidat broadcast
q2 = aggregate(orders.join(
    F.broadcast(customers),
    "customer_id", "inner"))
```

inner berarti key harus cocok di kedua sisi.

F.broadcast adalah perubahan yang diuji.

aggregate menghitung ringkasan wilayah.

collect menjalankan masing-masing query.

Notebook 8 / Script: dim; aggregate(orders.join(...))

©2025 Full Stack Bangalore

Hands-on join

Menjalankan perbandingan join

Notebook bagian 8 menjalankan kedua varian. Script memakai perintah berikut.

```
.\.venv\Scripts\python.exe lab\spark_lab.py `
  --experiment join `
  --rows 10000 --repeats 3 --hold-ui
```

--experiment: jenis percobaan.

--rows: jumlah transaksi buatan.

--repeats: jumlah trial tiap varian.

--hold-ui: menunggu Enter sebelum berhenti.

Notebook 8 / Script: --experiment join

©2025 Full Stack Bangalore

Hasil join

Contoh hasil pengujian notebook

Pengujian lokal: 10.000 baris, tiga trial per varian. Durasi dalam detik.

Varian	Median; rentang min–maks	Hasil
Sort-merge	0.2588; 0.2261–0.2727	Sesuai referensi
Broadcast	0.1294; 0.1070–0.1382	Sesuai referensi
Makna	Broadcast lebih cepat pada run ini.	Belum mewakili semua data

Notebook 8, 16 / Script: summary.json

©2025 Full Stack Bangalore

Pengaturan partisi

Repartition dan coalesce punya tujuan berbeda

Keduanya mengubah pembagian DataFrame, dengan cara dan biaya yang berbeda.

Operasi	Kapan digunakan	Hal yang diperhatikan
repartition(n, key)	Saat perlu membagi ulang data atau mengubah jumlah partisi.	Biasanya menambah shuffle.
coalesce(n)	Saat ingin mengurangi partisi, misalnya setelah data mengecil.	Pengurangan besar membatasi paralelisme.
Key tidak merata	Saat banyak baris memakai key sama.	repartition(key) belum tentu meratakan beban.

Notebook 9 / Script: konsep terkait partitions/skew

©2025 Full Stack Bangalore

Hands-on partisi

Dari empat partisi menjadi delapan lalu dua

Praktik ini tersedia pada notebook bagian 9.

```
repartitioned = direct.repartition(
    8, "customer_id")
coalesced = repartitioned.coalesce(2)

print(direct.rdd.getNumPartitions())
print(repartitioned.rdd.getNumPartitions())
print(coalesced.rdd.getNumPartitions())
```

direct: transaksi yang sudah memiliki region.

getNumPartitions membaca jumlah partisi.

Hasil yang diperiksa: 4, 8, lalu 2.

Agregasi sesudahnya harus tetap sama.

Notebook 9 / Script: tidak ada demo coalesce pada CLI

©2025 Full Stack Bangalore

Hands-on shuffle

Menguji 2, 8, dan 32 partisi shuffle

spark.sql.shuffle.partitions mengatur pembagian awal pada shuffle SQL.

```
spark.conf.set(
    "spark.sql.shuffle.partitions", "8")

# Perintah script:
# --experiment partitions
# --rows 10000 --repeats 3
```

Notebook bagian 10 membuat tiga varian.

Query join dan hasilnya tetap sama.

AQE dimatikan pada perbandingan ini.

Bandingkan median p2, p8, dan p32.

Notebook 10 / Script: --experiment partitions

©2025 Full Stack Bangalore

Optimasi saat berjalan

AQE menyesuaikan rencana saat data sudah terukur

AQE adalah Adaptive Query Execution.

Mengapa diperlukan

Rencana awal memakai informasi yang tersedia sebelum eksekusi.

Saat berjalan, Spark mengetahui ukuran hasil antara.

Yang dilihat pada lab

AQE dapat menggabungkan partisi shuffle yang kecil.

Kita membandingkan AQE mati dan aktif dari 64 partisi awal.

Notebook 11 / Script: --experiment aqe

©2025 Full Stack Bangalore

Hands-on AQE

Arti tulisan pada final plan AQE

Final plan adalah rencana setelah action selesai dijalankan.

AdaptiveSparkPlan	Query menggunakan mekanisme penyesuaian rencana.
AQEShuffleRead	Pembacaan hasil shuffle sudah mengikuti penyesuaian AQE.
isFinalPlan=true	Plan yang dibaca sudah merupakan rencana akhir untuk action itu.

Notebook 11 / Script: aqe_on_*_plan_after.txt

©2025 Full Stack Bangalore

Pemakaian ulang

Cache menyimpan hasil agar tidak dihitung ulang

Persist menyediakan pilihan cara penyimpanan. Cache memakai pilihan bawaan DataFrame.

Kapan membantu

Perhitungan awal cukup mahal. Hasilnya dipakai beberapa kali.

Contoh: tiga query memakai data yang sama.

Biaya yang dihitung

Materialisasi: mengerjakan dan mengisi cache pertama kali.

Pemakaian ulang: menjalankan query dari hasil yang disimpan.

Notebook 12 / Script: --experiment cache

©2025 Full Stack Bangalore

Data percobaan cache

Hash menambah perhitungan yang dipakai ulang

Hash mengubah nilai menjadi deretan karakter dengan panjang tertentu.

```
fingerprint = F.col("order_id").cast("string")
for _ in range(4):
    fingerprint = F.sha2(fingerprint, 256)
base = direct.withColumn(
    "fingerprint", fingerprint)

base = base.cache()
base.count()
```

cast("string") mengubah angka ke teks.

sha2(..., 256) menghasilkan hash SHA-256.

Empat putaran menjadi beban hitung buatan.

count memicu pengisian cache.

Notebook 12 / Script: fingerprint; base.cache()

©2025 Full Stack Bangalore

Hands-on cache

Perbandingan mencakup tiga query dan biaya awal

Notebook bagian 12 dan script cache memakai tiga kelompok transaksi.

Varian	Pekerjaan di dalam timer
Tanpa cache	Buat dan jalankan tiga query pada data dasar.
Dengan cache	Isi cache, lalu buat dan jalankan tiga query.
Hasil yang diperiksa	Penjualan per wilayah serta jumlah karakter fingerprint.
Sesudah pengukuran	Periksa hasil, kemudian unpersist melepaskan cache.

Notebook 12 / Script: --experiment cache

©2025 Full Stack Bangalore

Hasil cache

Contoh saat cache justru lebih lambat

Pengujian lokal: 10.000 baris, tiga trial untuk setiap varian.

Varian	Median total (detik)	Hasil
Tanpa cache	0.3000	Sesuai referensi
Dengan cache	0.3252	Sesuai referensi
Makna	Biaya awal belum terbayar pada run ini.	Ukur ulang pada beban yang relevan.

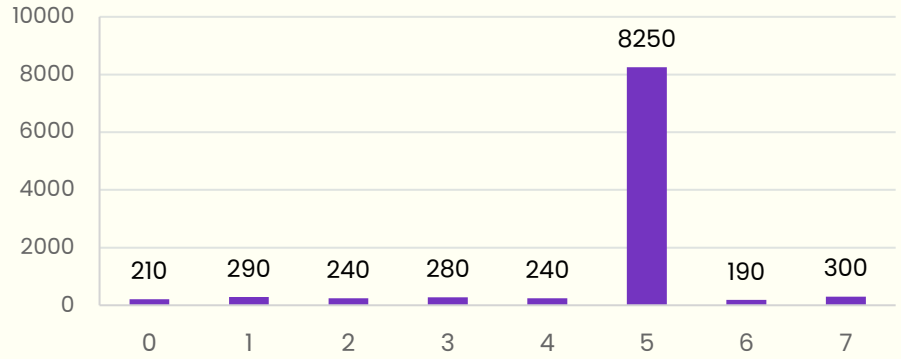
Notebook 12 / Script: summary.json untuk cache

©2025 Full Stack Bangalore

Data tidak merata

Skew: satu partisi menerima sebagian besar data

Grafik ini memakai hasil run notebook 10.000 baris, bukan angka ilustrasi.



Sumbu bawah: nomor partisi. Sumbu vertikal: jumlah baris; k berarti ribu.

Notebook 13 / Script: --experiment skew

Partisi 5 berisi 8.250 baris.

Partisi lain berisi 190–300 baris.

Jumlah semua partisi tetap 10.000.

©2025 Full Stack Bangalore

Hands-on skew

Mengapa satu key tetap menumpuk?

Kode memberi customer_id=0 kepada 80% baris sebelum repartition.

```
skewed = orders.withColumn("customer_id",
    F.when(F.col("order_id") % 10 < 8, 0)
     .otherwise(F.col("customer_id")))

parts = skewed.repartition(8, "customer_id")
```

when/otherwise memilih nilai berdasarkan kondisi.

Sisa 0–7 dari setiap 10 nomor menjadi key 0.

Key sama masuk partisi hash yang sama.

Demo menunjukkan skew, belum memperbaikinya.

Notebook 13 / Script: skewed; distribution

©2025 Full Stack Bangalore

Hands-on filter

Filter memilih baris; select memilih kolom

Contoh hanya memakai transaksi bernilai minimal 5.000.

```
filtered = orders.filter(
    F.col("amount") >= 5000
).select("order_id", "amount")

filtered.explain("formatted")
filtered.show(5, truncate=False)
```

filter membuang baris yang tidak memenuhi syarat.

select mempertahankan dua kolom.

Plan menampilkan Filter dan Project.

Notebook membandingkan count dengan Python.

Notebook 14 / Script: filter dipakai di cache/failure; demo ini khusus notebook

©2025 Full Stack Bangalore

Hands-on validasi

Kode sengaja menghilangkan satu wilayah

Pemeriksaan harus menangkap perbedaan, meskipun query berhasil dijalankan.

```
wrong = canonical(aggregate(
    direct.filter(F.col("region") != 0)
).collect())

try:
    assert wrong == reference
except AssertionError:
    print("Hasil berbeda: region 0 hilang")
```

!= berarti tidak sama dengan.

Assertion membandingkan isi hasil.

try/except menangani error yang diharapkan.

Notebook melanjutkan setelah error tertangkap.

Notebook 15 / Script: --experiment failure

©2025 Full Stack Bangalore

Membaca angka

Median, speedup, dan persentase waktu

Gunakan durasi yang diukur dari input dan pekerjaan yang sama.

Ukuran	Cara menghitung	Contoh hasil join
Median	Nilai tengah dari tiga waktu yang diurutkan.	Baseline 0,2588 detik
Speedup	Waktu baseline dibagi waktu kandidat.	2.00 kali
Penurunan durasi	(Baseline - kandidat) / baseline × 100%.	50.00%

Notebook 16 / Script: statistics.median(); summary

©2025 Full Stack Bangalore

Berkas keluaran

File mana yang dibuka setelah praktik?

JSON menyimpan data terstruktur. CSV menyimpan tabel yang dapat dibuka di spreadsheet.

Isi	Notebook	Script
Lingkungan / acuan	environment.json; reference.json	Nama sama
Ringkasan waktu	join_summary.json, cache_summary.json	summary.json per eksperimen
Rincian dan plan	all_trials.json; *_before/after.txt	trials.csv; *_plan_before/after.txt
Validasi selesai	verification.json	Status proses dan file tiap eksperimen

Notebook 3, 7–17 / Script: outputs/<waktu>_<eksperimen>

©2025 Full Stack Bangalore

Pilihan script

Enam eksperimen yang tersedia di terminal

Tambahkan pilihan berikut pada --experiment. Gunakan --rows 10000 untuk awal.

Pilihan	Yang dijalankan	Padanan notebook
join / partitions	Strategi join / 2, 8, 32 partisi shuffle	8 / 10
aqe / cache	AQE mati–aktif / pemakaian ulang hasil	11 / 12
skew / failure	Distribusi key timpang / kesalahan hasil	13 / 15
Tambahan notebook	repartition-coalesce dan filter-select langsung	9 / 14

Notebook 8–15 / Script: argparse choices

©2025 Full Stack Bangalore

Pemantauan

Spark UI: halaman untuk melihat pekerjaan

UI berarti antarmuka pengguna. Buka URL yang dicetak saat Spark mulai.

Tampilan / metrik	Yang dibaca
SQL	Query dan rencana eksekusinya.
Jobs / Stages	Daftar pekerjaan dan tahap yang dijalankan.
Duration / Input	Lama task / jumlah data yang dibaca.
Shuffle read / write	Data hasil shuffle yang dibaca / ditulis.

Notebook 4, 6–13 / Script: uiWebUrl; --hold-ui

©2025 Full Stack Bangalore

Menutup praktik

Cache, sesi Spark, dan environment

Penutupan yang benar membebaskan resource setelah latihan selesai.

```
# Pada akhir pemakaian cache
base.unpersist(blocking=True)

# Pada sel penutup notebook
spark.stop()
```

unpersist melepas data cache.

stop menghentikan sesi Spark latihan.

Notebook memulihkan environment proses.

UI berhenti; file hasil tetap tersedia.

Notebook 12, 17 / Script: finally: spark.stop()

©2025 Full Stack Bangalore

Saat ada kendala

Arti error yang sering muncul

Periksa lingkungan sebelum mengubah kode pengolahan data.

Pesan atau gejala	Arti	Langkah
No module named pyspark	Kernel belum memiliki PySpark.	Pilih kernel .venv lab.
Java tidak ditemukan	Java belum terlihat di PATH.	Periksa java -version.
SparkContext sudah aktif	Sesi lama masih ada.	Restart Kernel lalu Run All.
Waktu berubah-ubah	Beban komputer berbeda.	Baca median dan rentang trial.

Notebook 2; bagian kendala / Script: pemeriksaan environment

©2025 Full Stack Bangalore

Pemeriksaan akhir

Tanda seluruh notebook selesai

Status ini berasal dari pemeriksaan yang benar-benar dijalankan.

Pemeriksaan	Hasil run yang disertakan
Eksekusi sel	16 sel kode selesai pada kernel baru.
Pengukuran	27 trial: join, partisi shuffle, AQE, dan cache.
Kebenaran	Referensi, skew, filter, dan kesalahan sengaja diperiksa.
Penutupan	completed=true dan spark_stopped=true.

Notebook 17 / Script: lihat status masing-masing eksperimen

©2025 Full Stack Bangalore

Peta materi

Hubungan PPT, notebook, dan script

Nomor bagian notebook tercantum di bawah setiap slide.

Kelompok materi	Notebook	Script
Persiapan, data, dan rencana	2–7	setup; orders/customers; save_plan
Join, partisi, dan AQE	8–11	join; partitions; aqe
Cache, skew, filter, validasi	12–15	cache; skew; failure
Ringkasan dan penutupan	16–17	summary; finally

Notebook 1–17 / Script: seluruh jalur utama

©2025 Full Stack Bangalore